

Lernende Systeme

Lernen von Akzeptoren

Thema 7

24.04.2026
Fischer

Zielsetzung

- Iteratives Lernen eines deterministischen endlichen Automaten durch eine wachsende Menge positiver Beispiele
- Formale Darstellung und grafische Ausgabe
- Möglichkeit zum Testen der Hypothese auf weitere Wörter

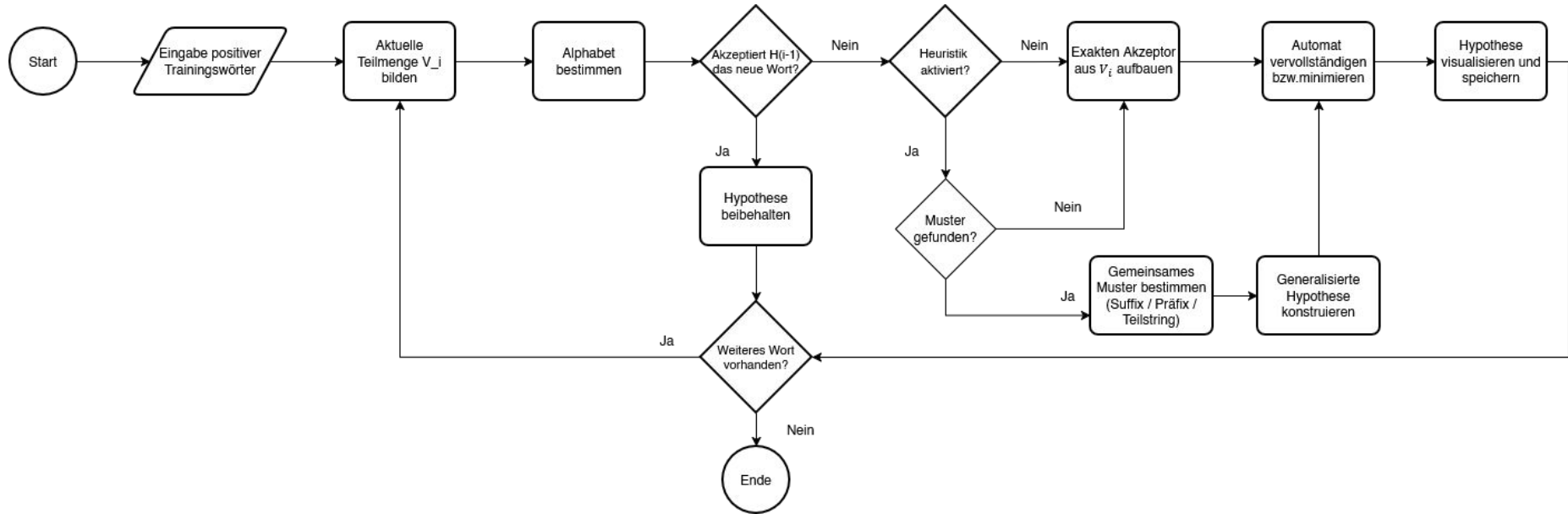
Implementierung Aufbau

- Python
- Visualisierung des Automaten mit Graphviz

acceptor.py: Kernlogik des Akzeptors

app.py: Ablaufsteuerung des Lernprozesses mit Streamlit-Oberfläche

Implementierung Ablauf



Implementierung Detail

Konstruktion der exakten Hypothese

```
527         if learner is None:
528             learner = Acceptor()
529             for w in observed:
530                 learner.learn_word(w)
531             learner.Sigma = sigma
532             learner.minimize()
533             ensure_total(learner)
534             lang_mode = "exact"
```

529 Präfixbaum aus allen bisherigen
Wörtern aufbauen

532 Minimierung

533 Vollständigkeit absichern

Implementierung Detail

Neues Wort dem Präfixbaum hinzufügen - learn_word

```
14 def learn_word(self, word):
15     """
16     Inserts a word into the prefix tree.
17     """
18     current = self.q0
19     for ch in word:
20         self.Sigma.add(ch)
21         needs_new_edge = (current, ch) not in self.delta
22         # allow overriding a previous dead-state edge when learning new positive samples
23         if not needs_new_edge and self.dead_state is not None:
24             needs_new_edge = self.delta[(current, ch)] == self.dead_state
25
26         if needs_new_edge:
27             new_state = self.next_state_id
28             self.Q.add(new_state)
29             self.delta[(current, ch)] = new_state
30             self.next_state_id += 1
31         current = self.delta[(current, ch)]
32     self.F.add(current)
```

- 19 Wort wird zeichenweise eingelesen
- 20 jedes neue Zeichen wird ins Alphabet aufgenommen
- 21 es wird geprüft, ob es für dieses Zeichen vom aktuellen Zustand schon einen Übergang gibt
 - wenn nicht:
 - 26-30 neuer Zustand erzeugen, neuer Übergang eintragen
- 32 Endzustand des Wortes wird als akzeptierend markiert

Implementierung Detail

Minimierung - minimize

```
46 def minimize(self, make_total=True):  
47     if make_total:  
48         self._make_total()  
49  
50     partitions = self._hopcroft_partitions()  
51     self._build_from_partitions(partitions)  
52     self._remove_unreachable()  
53     self.next_state_id = (max(self.Q) + 1) if self.Q else 0
```

48 Automaten vervollständigen
50 Hopcroft Partition berechnen
51 Neuen Automaten aus Partition bauen
52 Unerreichbare Zustände entfernen
53 Nummerierung anpassen

Implementierung Detail

Hopcroft Partitionierung - hopcroft_partitions

```
158 def _hopcroft_partitions(self):
159     alphabet = sorted(self.Sigma)
160     F = set(self.F)
161     NF = self.Q - F
162     partitions = [F, NF] if NF else [F]
163     # remove empties
164     partitions = [p for p in partitions if p]
165
166     waiting = partitions.copy()
167     while waiting:
168         A = waiting.pop()
169         for c in alphabet:
170             # states that transition via c into A
171             X = {q for q in self.Q if (q, c) in self.delta and self.delta[(q, c)] in A}
172             new_partitions = []
173             for Y in partitions:
174                 inter = Y & X
175                 diff = Y - X
176                 if inter and diff:
177                     new_partitions.extend([inter, diff])
178                     if Y in waiting:
179                         waiting.remove(Y)
180                         waiting.extend([inter, diff])
181                     else:
182                         waiting.append(inter if len(inter) <= len(diff) else diff)
183             else:
184                 new_partitions.append(Y)
185         partitions = new_partitions
186     return partitions
```

159 Alphabet wird sortiert
160-164 Zustände werden in akzeptierend und nicht akzeptierend getrennt -> Anfangspartitionen werden daraus erstellt
167 Partitionen aus der Warteliste werden nacheinander bearbeitet
171 Haben Partitionen das gleiche Verhalten?
Für jedes Zeichen wird die Menge X bestimmt -> alle Zustände die mit diesem Zeichen in die aktuelle Partition übergehen
173 - 177 Zeigt eine Partition dabei unterschiedliches Verhalten, wird sie in zwei Partitionen aufgespalten

Demonstration

Grenzen des Systems

- Ohne negative Beispiele ist eine exakte Identifikation der Zielsprache im Allgemeinen nicht eindeutig.
- Bei der Generalisierung kann es dazu kommen, dass auch Wörter akzeptiert werden, die nicht zur Sprache L gehören.
- Die Generalisierung basiert nur auf einfachen Heuristiken, etwa gemeinsamen Suffixen, Präfixen oder Teilstrings.